

Documentação do Projeto de Dados — TikTok + TikTok Shop

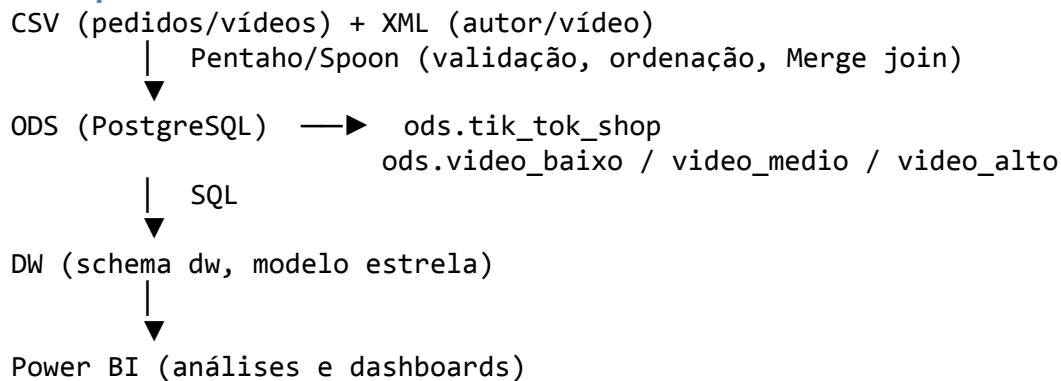
1. Visão geral

Projeto de dados analíticos que integra o **desempenho de conteúdo do TikTok** com as **vendas e pedidos do TikTok Shop**. Dois conjuntos de dados (conteúdo/engajamento e vendas/pedidos) foram unificados, e o elo entre eles é o campo **id_video**: cada pedido pode ser associado ao vídeo responsável.

A fonte unificada no ODS é **ods.tik_tok_shop** (pedido + vídeo/autor na mesma tabela). Dela, os dados seguem para um Data Warehouse em modelo estrela.

Objetivo: analisar não só o desempenho individual dos vídeos, mas a relação entre conteúdo, engajamento e resultado comercial.

2. Arquitetura



Camada	Ferramenta	Observação
Extração/transformação	Pentaho/Spoon	Leitura de CSV e XML, Data validator, Sort rows, Merge join
ODS	PostgreSQL (pgAdmin)	Fonte unificada e tabelas por faixa de desempenho
DW	SQL	Schema dw, modelo dimensional em estrela
Visualização	Power BI	Etapa posterior

3. Dados utilizados

3.1 Conteúdo e engajamento (vídeos)

ID do vídeo; ID e nome do autor; categoria; descrição; país; data de publicação; duração; visualizações; curtidas; comentários; compartilhamentos; salvamentos; seguidores do autor; hashtags; classificação/faixa de desempenho; gênero e faixa etária do público.

3.2 Vendas e pedidos

ID do pedido; ID do vídeo relacionado; produto; categoria do produto; preço unitário; quantidade; valor total; desconto; uso de cupom; método de pagamento; status do pedido; avaliação; data do pedido.

4. Integração conteúdo ↔ vendas

- Chave de ligação: **id_video**.
- Na tabela fato, o pedido recebe a referência **sk_video** da dimensão de vídeos.
- A associação é feita na carga por um **LEFT JOIN** entre os pedidos e **dw.dim_video**. O **LEFT JOIN** mantém pedidos mesmo sem vídeo correspondente.

Perguntas analíticas possibilitadas

- Quais vídeos estão associados a mais pedidos?
- Quais conteúdos têm maior volume de vendas?
- Quais categorias de produtos aparecem nos pedidos?
- Qual método de pagamento é mais usado?
- Como os pedidos se distribuem ao longo do tempo?
- Qual o status dos pedidos?
- Existe informação de engajamento associada aos pedidos?

Estas são possibilidades de análise da estrutura criada, não resultados. Os resultados serão apresentados no Power BI.

5. Passo a passo da transformação no Pentaho (Spoon)

A transformação **transformação tik_tokshop** está organizada em três blocos (anotações no canvas): **PUBLIC**, **ODS** e **TRATAMENTO**.

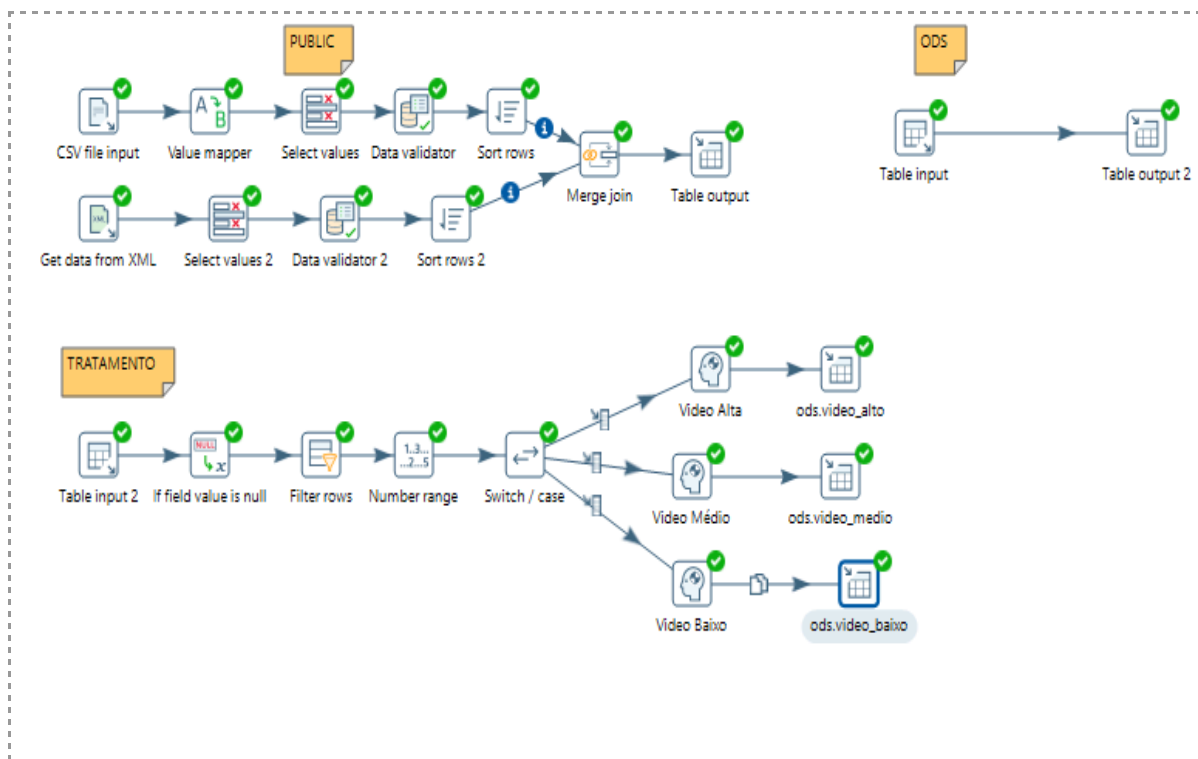


Figura 1 — Transformação completa no Spoon, com os blocos PUBLIC, ODS e TRATAMENTO.

5.1 Bloco PUBLIC — leitura, validação e junção das fontes

O nome PUBLIC é só uma anotação do canvas para o bloco. Não é uma camada do projeto: as tabelas dessa etapa ficam no schema public porque esse é o padrão do PostgreSQL.

Dois fluxos paralelos, que se encontram em um **Merge join**.

Fluxo 1 — vídeos (CSV)

#	Step	Função
1	CSV file input	Lê o arquivo CSV com os dados dos vídeos
2	Value mapper	Mapeia/padroniza valores de campos
3	Select values	Seleciona, renomeia e ajusta os campos que seguem no fluxo
4	Data validator	Valida os dados
5	Sort rows	Ordena as linhas por id_video (a chave do Merge join, que exige entradas ordenadas)

Fluxo 2 — pedidos (XML)

#	Step	Função
1	Get data from XML	Lê o arquivo XML com os dados dos pedidos
2	Select values 2	Seleciona, renomeia e ajusta os campos
3	Data validator 2	Valida os pedidos: id_pedido (não pode ser nulo), preço unitário, valor total, quantidade, desconto percentual e cupom. Reporta todos os erros e concatena em uma linha, separados por “;”
4	Sort rows 2	Ordena as linhas por id_video

Junção e saída

#	Step	Função
1	Merge join	Junta vídeos e pedidos pela chave id_video, em modo LEFT OUTER: cada pedido recebe os dados do seu vídeo, e os vídeos sem pedido são mantidos, com os campos do pedido vazios
2	Table output	Grava o resultado unificado em uma tabela de passagem no schema public (com a opção “Truncate table” marcada, para cada execução substituir os dados em vez de acrescentar)

Por que Merge join e não Stream lookup: a primeira versão usava um Stream lookup, que devolve só uma linha por chave. Como um vídeo tem vários pedidos, apenas um pedido por vídeo era mantido (411 pedidos em vez de 1.524) e os demais se perdiam. O Merge join

mantém todas as combinações, e por isso exige que as duas entradas estejam ordenadas pela chave.

5.2 Bloco ODS — carga para o ODS

#	Step	Função
1	Table input	Lê a tabela unificada de origem
2	Table output 2	Grava em ods.tik_tok_shop

5.3 Bloco TRATAMENTO — classificação de desempenho dos vídeos

#	Step	Função
1	Table input 2	Lê os dados do ODS
2	If field value is null	Trata valores nulos (define valor padrão)
3	Filter rows	Filtra as linhas que seguem no fluxo
4	Number range	Cria as faixas numéricas de desempenho
5	Switch / case	Direciona cada linha conforme a faixa

O Switch / case abre três ramos:

Ramo	Step	Destino
Alta	Video Alta	ods.video_alto
Média	Video Médio	ods.video_medio
Baixa	Video Baixo	ods.video_baixo

5.4 Resumo do fluxo

PUBLIC: CSV → Value mapper → Select values → Data validator → Sort rows

└─ Merge join → Table output
XML → Select values 2 → Data validator 2 → Sort rows 2

ODS: Table input → Table output 2 (ods.tik_tok_shop)

TRATAMENTO: Table input 2 → If field value is null → Filter rows → Number range → Switch/case

Alta → ods.video_alto

Médio → ods.video_medio

└─ Video

└─ Video

Baixo → ods.video_baixo

Truncate table: todos os Table output do fluxo (public, ODS e as três tabelas de vídeo) usam a opção “Truncate table”. Assim, rodar a transformação mais de uma vez não duplica as linhas.

Ordem de execução: os três blocos dependem um do outro, porque cada um lê a tabela gravada pelo anterior (o ODS lê a tabela do public, e o TRATAMENTO lê o ODS). Por isso devem rodar em sequência: primeiro PUBLIC, depois ODS, depois TRATAMENTO. Se rodarem juntos no mesmo arquivo, os steps começam ao mesmo tempo e o bloco seguinte pode ler a tabela ainda incompleta.

6. Data Warehouse (schema dw)

O DW é criado por um script SQL no PostgreSQL (pgAdmin), em modelo dimensional em estrela. O script cria o schema dw e, em blocos numerados, cada dimensão e a tabela fato. As dimensões são carregadas a partir da fonte unificada ods.tik_tok_shop.

Todas as dimensões seguem o mesmo padrão: uma **chave substituta** (sk_..., do tipo SERIAL PRIMARY KEY), gerada automaticamente pelo banco, e a **chave natural** vinda da fonte (por exemplo id_video), marcada como UNIQUE para impedir duplicidade.

6.1 Dimensão Tempo — dw.dim_tempo

Bloco “1. DIMENSÃO TEMPO” do script.

Coluna	Tipo	Descrição
sk_tempo	SERIAL PRIMARY KEY	Chave substituta
data	DATE NOT NULL UNIQUE	Data do calendário (uma linha por dia)
dia	INT	Dia do mês
mes	INT	Mês
trimestre	INT	Trimestre (1 a 4)
ano	INT	Ano
dia_semana	VARCHAR(15)	Nome do dia da semana

Como é carregada: um INSERT INTO ... SELECT usa generate_series('2024-01-01'::date, '2027-12-31'::date, interval '1 day') para gerar uma linha para cada dia desse intervalo (1.461 dias). Para cada data d, a função EXTRACT separa dia, mês, trimestre (QUARTER) e ano, e TO_CHAR(d, 'FM Day') devolve o nome do dia da semana (o FM remove os espaços de preenchimento do nome).

Por que existe: permite analisar os pedidos por dia, mês, trimestre, ano e dia da semana sem calcular isso a cada consulta.

```

-- 1. DIMENSÃO TEMPO

CREATE TABLE dw.dim_tempo (
    sk_tempo        SERIAL PRIMARY KEY,
    data            DATE NOT NULL UNIQUE,
    dia            INT,
    mes            INT,
    trimestre      INT,
    ano            INT,
    dia_semana     VARCHAR(15)
);

INSERT INTO dw.dim_tempo (data, dia, mes, trimestre, ano, dia_semana)
SELECT d::date,
       EXTRACT(DAY FROM d),
       EXTRACT(MONTH FROM d),
       EXTRACT(QUARTER FROM d),
       EXTRACT(YEAR FROM d),
       TO_CHAR(d, 'FMDay')
FROM generate_series('2024-01-01'::date, '2027-12-31'::date, interval '1 day') d;

```

Figura 2 — Script de criação e carga da dim_tempo.

6.2 Dimensão Vídeo — dw.dim_video

Bloco “2. DIMENSÃO VÍDEO” do script. Concentra as características e as métricas dos vídeos e de seus autores, com **um registro por id_video**.

Grupo	Colunas	Tipo
Chaves	sk_video	SERIAL PRIMARY KEY
	id_video	VARCHAR(50) NOT NULL UNIQUE
Autor	id_autor	VARCHAR(50)
	autor	VARCHAR(150)
Descrição	categoria, pais	VARCHAR(100)
	descricao, hashtags	TEXT
	data_publicacao	DATE
Métricas	duracao_segundos,	NUMERIC
	views, likes,	
	comentarios,	
	compartilhamentos,	
	salvamentos,	
	seguidores_autor	

Grupo	Colunas	Tipo
Classificação e público	faixa_engajamento, genero_publico, faixa_etaria_publico	VARCHAR(30)

```
-- 2. DIMENSÃO VÍDEO

CREATE TABLE dw.dim_video (
    sk_video          SERIAL PRIMARY KEY,
    id_video          VARCHAR(50) NOT NULL UNIQUE,
    id_autor          VARCHAR(50),
    autor             VARCHAR(150),
    categoria         VARCHAR(100),
    descricao         TEXT,
    pais              VARCHAR(100),
    data_publicacao   DATE,
    duracao_segundos  NUMERIC,
    views             NUMERIC,
    likes             NUMERIC,
    comentarios       NUMERIC,
    compartilhamentos NUMERIC,
    salvamentos       NUMERIC,
    seguidores_autor  NUMERIC,
    hashtags          TEXT,
    faixa_engajamento VARCHAR(30),
    genero_publico    VARCHAR(30),
    faixa_etaria_publico VARCHAR(30)
);
```

Figura 3 — Criação da tabela dim_video.

Como é carregada: o `INSERT INTO dw.dim_video (...)` recebe o resultado de um `SELECT DISTINCT ON (id_video)` sobre `ods.tik_tok_shop`:

- **DISTINCT ON (id_video):** a fonte tem uma linha por pedido, então o mesmo vídeo aparece várias vezes. Essa cláusula mantém apenas uma linha por vídeo, garantindo um registro por id_video (o `ORDER BY id_video` é exigido por esse comando). Como não há critério de desempate, quando um vídeo aparece em mais de uma linha o banco escolhe uma delas.
- **WHERE id_video IS NOT NULL:** descarta linhas sem identificador de vídeo.
- **ORDER BY id_video:** ordena o resultado e atende à exigência do `DISTINCT ON`.
- **Coluna de classificação:** a coluna `classificacao_desempenho` da fonte (Alto, Médio, Baixo) é carregada na coluna `faixa_engajamento` da dimensão. As colunas

do INSERT e do SELECT são associadas pela posição, então os nomes diferentes são esperados.

```
INSERT INTO dw.dim_video (id_video, id_autor, autor, categoria, descricao, pais,
                          data_publicacao, duracao_segundos, views, likes,
                          comentarios, compartilhamentos, salvamentos,
                          seguidores_autor, hashtags, faixa_engajamento,
                          genero_publico, faixa_etaria_publico)

SELECT DISTINCT ON (id_video)
    id_video, id_autor, autor, categoria, descricao, pais,
    data_publicacao, duracao_segundos, views, likes,
    comentarios, compartilhamentos, salvamentos,
    seguidores_autor, hashtags, classificacao_desempenho,
    genero_publico, faixa_etaria_publico
FROM ods.tik_tok_shop
WHERE id_video IS NOT NULL
ORDER BY id_video;
```

Figura 4 — Carga da dim_video (INSERT ... SELECT DISTINCT ON) e início do bloco da dim_metodo_pagamento.

6.3 Dimensão Método de Pagamento — dw.dim_metodo_pagamento

Bloco “3. DIMENSÃO MÉTODO DE PAGAMENTO” do script.

Coluna	Tipo	Descrição
sk_metodo_pagamento	SERIAL PRIMARY KEY	Chave substituta
metodo_pagamento	VARCHAR(50) UNIQUE	Nome do método de pagamento

```
CREATE TABLE dw.dim_metodo_pagamento (
    sk_metodo_pagamento SERIAL PRIMARY KEY,
    metodo_pagamento    VARCHAR(50) UNIQUE
);
```

```
INSERT INTO dw.dim_metodo_pagamento (metodo_pagamento)
SELECT DISTINCT metodo_pagamento
FROM ods.tik_tok_shop
WHERE metodo_pagamento IS NOT NULL;
```

Explicação: tabela pequena, carregada com os valores distintos de método de pagamento da fonte (SELECT DISTINCT), ignorando os nulos (WHERE ... IS NOT NULL). Em vez de repetir o texto do método em cada pedido, a tabela fato guarda apenas a chave sk_metodo_pagamento.

6.4 Dimensão Status do Pedido — dw.dim_status_pedido

Bloco “4. DIMENSÃO STATUS DO PEDIDO” do script. Segue exatamente o mesmo padrão da dimensão de método de pagamento.

Coluna	Tipo	Descrição
sk_status_pedido	SERIAL PRIMARY KEY	Chave substituta
status_pedido	VARCHAR(50) UNIQUE	Status do pedido

```
CREATE TABLE dw.dim_status_pedido (  
    sk_status_pedido SERIAL PRIMARY KEY,  
    status_pedido    VARCHAR(50) UNIQUE  
);
```

```
INSERT INTO dw.dim_status_pedido (status_pedido)  
SELECT DISTINCT status_pedido  
FROM ods.tik_tok_shop  
WHERE status_pedido IS NOT NULL;
```

Explicação: carrega os status distintos encontrados na fonte, sem nulos, e a fato guarda só a chave sk_status_pedido.

```

-- 3. DIMENSÃO MÉTODO DE PAGAMENTO

CREATE TABLE dw.dim_metodo_pagamento (
    sk_metodo_pagamento SERIAL PRIMARY KEY,
    metodo_pagamento VARCHAR(50) UNIQUE
);

INSERT INTO dw.dim_metodo_pagamento (metodo_pagamento)
SELECT DISTINCT metodo_pagamento
FROM ods.tik_tok_shop
WHERE metodo_pagamento IS NOT NULL;

-- =====
-- 4. DIMENSÃO STATUS DO PEDIDO

CREATE TABLE dw.dim_status_pedido (
    sk_status_pedido SERIAL PRIMARY KEY,
    status_pedido VARCHAR(50) UNIQUE
);

INSERT INTO dw.dim_status_pedido (status_pedido)
SELECT DISTINCT status_pedido
FROM ods.tik_tok_shop
WHERE status_pedido IS NOT NULL;

```

Figura 5 — Dimensões método de pagamento e status do pedido.

6.5 Tabela fato — dw.fact_pedidos

Bloco “5. TABELA FATO - PEDIDOS” do script. É a estrutura central para a análise dos pedidos: cada linha representa um pedido.

Grupo	Coluna	Tipo
Chave	sk_pedido	SERIAL PRIMARY KEY
	id_pedido	VARCHAR(50)
Chaves estrangeiras	sk_video	INT, referencia dw.dim_video
	sk_tempo	INT, referencia dw.dim_tempo
	sk_metodo_pagamento	INT, referencia dw.dim_metodo_pagamento
	sk_status_pedido	INT, referencia dw.dim_status_pedido
Produto	nome_produto	VARCHAR(150)
	categoria_produto	VARCHAR(100)

Grupo	Coluna	Tipo
Medidas	preco_unitario_brl	NUMERIC(10,2)
	quantidade	INT
	valor_total_brl	NUMERIC(12,2)
	desconto_percentual	NUMERIC(5,2)
	cupom_aplicado	VARCHAR(5)
	avaliacao	NUMERIC(3,1)
	taxa_engajamento	NUMERIC(6,2)

```
CREATE TABLE dw.fact_pedidos (
    sk_pedido          SERIAL PRIMARY KEY,
    id_pedido          VARCHAR(50),
    sk_video            INT REFERENCES dw.dim_video(sk_video),
    sk_tempo            INT REFERENCES dw.dim_tempo(sk_tempo),
    sk_metodo_pagamento INT REFERENCES
dw.dim_metodo_pagamento(sk_metodo_pagamento),
    sk_status_pedido   INT REFERENCES
dw.dim_status_pedido(sk_status_pedido),
    nome_produto        VARCHAR(150),
    categoria_produto   VARCHAR(100),
    preco_unitario_brl  NUMERIC(10,2),
    quantidade          INT,
    valor_total_brl     NUMERIC(12,2),
    desconto_percentual NUMERIC(5,2),
    cupom_aplicado      VARCHAR(5),
    avaliacao           NUMERIC(3,1),
    taxa_engajamento   NUMERIC(6,2)
);
```

Explicação da criação: as quatro colunas sk_... são chaves estrangeiras (REFERENCES) que ligam a fato às dimensões, formando a estrela. As demais colunas são os dados comerciais e as medidas do pedido. Os valores monetários usam NUMERIC com casas decimais definidas (por exemplo, NUMERIC(12,2) aceita até 10 dígitos antes da vírgula e 2 depois).

```
CREATE TABLE dw.fact_pedidos (
  sk_pedido          SERIAL PRIMARY KEY,
  id_pedido          VARCHAR(50),
  sk_video           INT REFERENCES dw.dim_video(sk_video),
  sk_tempo           INT REFERENCES dw.dim_tempo(sk_tempo),
  sk_metodo_pagamento INT REFERENCES dw.dim_metodo_pagamento(sk_metodo_pagamento),
  sk_status_pedido   INT REFERENCES dw.dim_status_pedido(sk_status_pedido),
  nome_produto       VARCHAR(150),
  categoria_produto   VARCHAR(100),
  preco_unitario_brl  NUMERIC(10,2),
  quantidade          INT,
  valor_total_brl     NUMERIC(12,2),
  desconto_percentual NUMERIC(5,2),
  cupom_aplicado      VARCHAR(5),
  avaliacao           NUMERIC(3,1),
  taxa_engajamento   NUMERIC(6,2)
);
```

Figura 6 — Criação da tabela fato dw.fact_pedidos.

```
INSERT INTO dw.fact_pedidos (id_pedido, sk_video, sk_tempo,
                             sk_metodo_pagamento,
                             sk_status_pedido, nome_produto,
                             categoria_produto,
                             preco_unitario_brl, quantidade, valor_total_brl,
                             desconto_percentual, cupom_aplicado, avaliacao,
                             taxa_engajamento)

SELECT p.id_pedido,
       v.sk_video,
       t.sk_tempo,
       mp.sk_metodo_pagamento,
       sp.sk_status_pedido,
       p.nome_produto,
       p.categoria_produto,
       p.preco_unitario_brl,
       p.quantidade,
       p.valor_total_brl,
       p.desconto_percentual,
       p.cupom_aplicado,
       p.avaliacao,
       p.taxa_engajamento
FROM ods.tik_tok_shop p
LEFT JOIN dw.dim_video v      ON v.id_video = p.id_video
LEFT JOIN dw.dim_tempo t      ON t.data = p.data_pedido::date
LEFT JOIN dw.dim_metodo_pagamento mp ON mp.metodo_pagamento =
p.metodo_pagamento
LEFT JOIN dw.dim_status_pedido sp  ON sp.status_pedido = p.status_pedido;
```

Explicação da carga: o SELECT parte de ods.tik_tok_shop (alias p, uma linha por pedido) e troca cada atributo descritivo pela chave substituta da dimensão correspondente:

- dim_video: ligada pelo id_video;

- `dim_tempo`: ligada pela data do pedido (`data_pedido::date` converte o campo para data);
- `dim_metodo_pagamento`: ligada pelo texto do método de pagamento;
- `dim_status_pedido`: ligada pelo texto do status.

Todos os relacionamentos usam **LEFT JOIN**, então nenhum pedido é perdido: se não houver correspondência (por exemplo, pedido sem vídeo ou com data fora do período da `dim_tempo`, que vai de 2024 a 2027), a chave correspondente fica nula e o pedido continua na fato. As demais colunas (produto, preços, quantidade, valores, desconto, cupom, avaliação e taxa de engajamento) são copiadas diretamente da fonte.

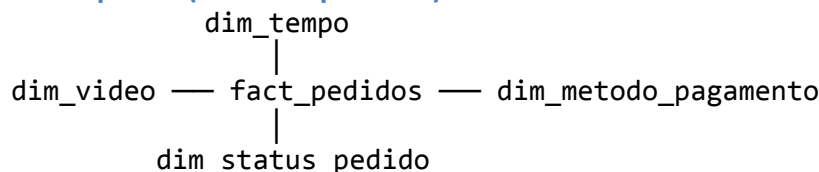
Vídeos sem pedido: a fonte `ods.tik_tok_shop` também traz os vídeos que não têm compra, e o **INSERT** não filtra essas linhas. A fato guarda, então, uma linha para cada vídeo sem pedido, com `id_pedido`, produto, valores, data, status e pagamento vazios. Isso foi mantido de propósito, para que os vídeos sem vendas também apareçam na análise.

```
INSERT INTO dw.fact_pedidos (id_pedido, sk_video, sk_tempo, sk_metodo_pagamento, sk_status_pedido,
                             nome_produto, categoria_produto, preco_unitario_brl, quantidade,
                             valor_total_brl, desconto_percentual, cupom_aplicado, avaliacao,
                             taxa_engajamento)

SELECT p.id_pedido,
       v.sk_video,
       t.sk_tempo,
       mp.sk_metodo_pagamento,
       sp.sk_status_pedido,
       p.nome_produto,
       p.categoria_produto,
       p.preco_unitario_brl,
       p.quantidade,
       p.valor_total_brl,
       p.desconto_percentual,
       p.cupom_aplicado,
       p.avaliacao,
       p.taxa_engajamento
FROM   ods.tik_tok_shop p
LEFT JOIN dw.dim_video v       ON v.id_video = p.id_video
LEFT JOIN dw.dim_tempo t       ON t.data = p.data_pedido::date
LEFT JOIN dw.dim_metodo_pagamento mp ON mp.metodo_pagamento = p.metodo_pagamento
LEFT JOIN dw.dim_status_pedido sp  ON sp.status_pedido = p.status_pedido;
```

Figura 7 — Carga da tabela fato (**INSERT ... SELECT** com **LEFT JOINS**).

6.6 Esquema (visão simplificada)



7. Dashboard no Power BI — TIK TOK ANALYTICS

Com o DW pronto, os dados foram levados ao Power BI para construir o dashboard **TIK TOK ANALYTICS**, um painel de desempenho e engajamento que reúne vídeos, conteúdo, público e vendas. Ele responde, de forma visual, às perguntas analíticas listadas na seção 4.

7.1 Navegação e filtros

O dashboard é dividido em páginas, acessadas pelo menu lateral escuro presente em todas as telas:

Página	Foco
Capa	Apresentação do dashboard
01 Visão Geral	Indicadores gerais de vendas e desempenho por engajamento, produto, tipo de conteúdo e trimestre
02 Conteúdo & Engajamento	Desempenho dos vídeos, categorias e criadores
03 Pagamentos & Pedidos	Evolução mensal dos pedidos e distribuição por status
04 Relatório Geral	Visão consolidada de conteúdo, produtos e criadores, com atalhos para o plano de marketing (Canva) e para documentos (Google)

No canto superior direito das páginas há um **filtro de mês** (controle deslizante de 1 a 12) que permite recortar o período analisado. Esse filtro vem da dimensão tempo (`dw.dim_tempo`).

7.2 Capa

Tela de abertura com o título **TIK TOK ANALYTICS** e a descrição “Dashboard de desempenho e engajamento: análise completa do desempenho de vídeos, campanha, conteúdo e público para transformar dados em estratégia e resultado”. Os ícones na parte inferior representam os temas abordados: vídeos, desempenho, objetivo/meta, público e alcance geográfico.



Figura 8 — Capa do dashboard.

7.3 Página 01 — Visão Geral

Resume o desempenho comercial e o engajamento em uma única tela.

Cartões de indicadores (KPIs):

Indicador	Valor exibido	O que representa
Total por pedido	1,613 Mil	Total de registros da tabela fato (1.524 pedidos mais 89 vídeos sem pedido)
Valor total	199,04 Mil	Soma do valor total dos pedidos (valor_total_br1)
Desconto por porcentagem	5,55 Mil	Soma do desconto percentual (desconto_percentual)
Valor por engajamento	24,34 Mil	Soma da taxa de engajamento (taxa_engajamento)
Média de avaliação	4,09	Média das notas de avaliação dos pedidos (avaliação)

Gráficos:

- **Soma por engajamento** (linha): compara as três faixas de desempenho dos vídeos (Alto, Médio e Baixo). A curva é decrescente: a faixa Alto soma cerca de 680, a Médio cerca de 130 e a Baixo cerca de 50.
- **Produtos com Menos Pedidos** (barras horizontais): mostra os cinco produtos com menos pedidos e a quantidade de cada um: Batom Matte Rosa (cerca de 70), Creme Hidratante Facial (cerca de 64), Tênis Corrida Pro (cerca de 51), Faixa Elástica Fitness (cerca de 43) e Whey Protein Chocolate (cerca de 37).
- **Contagem por tipo de Conteúdo** (rosca): distribui a contagem entre cinco categorias de conteúdo, com quantidade e percentual: Comédia (198; 22,81%), Dança (186; 21,43%), Educação (172; 19,82%), Entretenimento (162; 18,66%) e Lifestyle (150; 17,28%). O total exibido no centro é 868.
- **Valor Total por Trimestre** (colunas): vendas de cerca de 85 Mil no trimestre 1, cerca de 85 Mil no trimestre 2 e cerca de 28 Mil no trimestre 3. Os pedidos vão de janeiro a julho de 2026, então o trimestre 3 tem apenas o mês de julho: é um trimestre incompleto, não uma queda real.

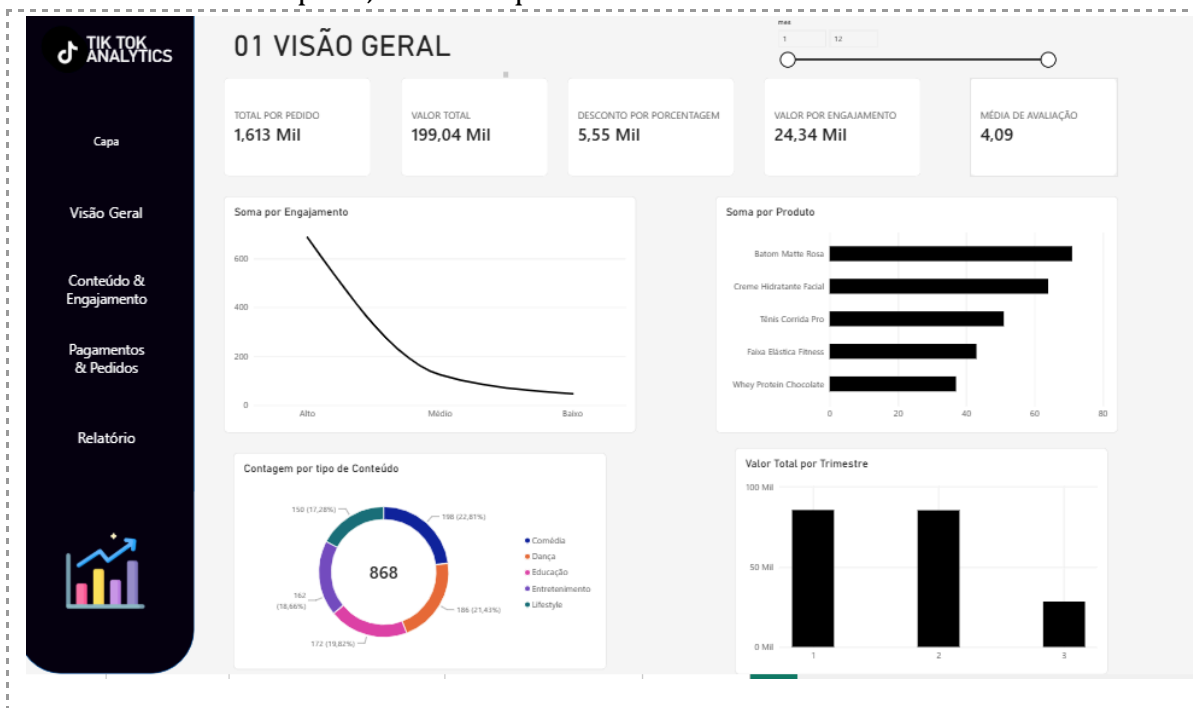


Figura 9 — Página 01 Visão Geral.

7.4 Página 02 — Conteúdo & Engajamento

Foca no desempenho dos vídeos e de seus criadores, usando principalmente a dimensão `dw.dim_video`.

Cartões de indicadores (KPIs):

Indicador	Valor exibido	O que representa
Total de vídeos	500	Quantidade de vídeos distintos
Total de likes	3,03 Bi	Total de curtidas
Total de views	30,35 Bi	Total de visualizações
Média de engajamento	15,09	Taxa média de engajamento

Gráficos e tabela:

- **Engajamento por Conteúdo** (rosca): distribui o engajamento (total de 12,83 Mil) entre as categorias Comédia, Educação, Lifestyle, Beleza e Desafio.
- **Engajamento por Criador de Conteúdo** (barras horizontais): ranking dos criadores com maior engajamento: fitnessbrasil, charlidamelio, zachking, spencerx e mrbeast.
- **Tabela de vídeos**: lista cada vídeo com autor, views, likes e soma da taxa de engajamento, com total geral de 17.351,94 no rodapé. Serve para detalhar e comparar vídeos individualmente.
- **Vídeos com Pedidos por Mês** (linha): quantidade de vídeos distintos que tiveram pelo menos um pedido em cada mês, de 1 a 7: 159, 178, 173, 161, 169, 170 e 159. O mês vem da data do pedido, e não da data das visualizações, que não existe na tabela fato.

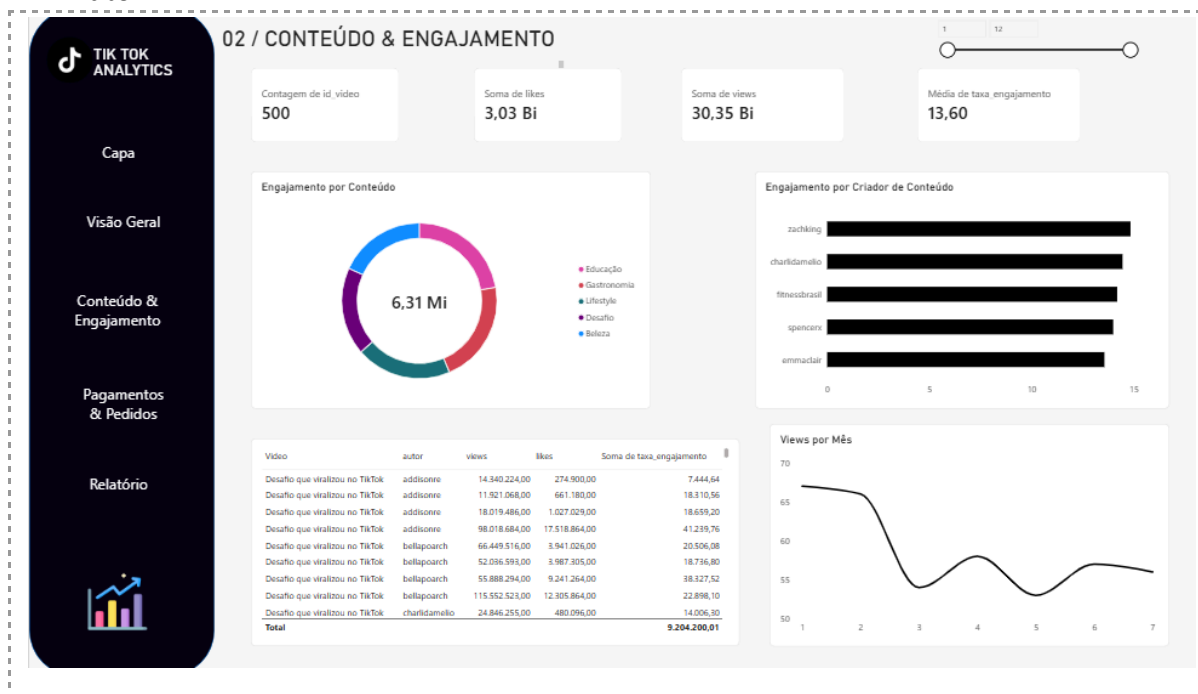


Figura 10 — Página 02 Conteúdo & Engajamento.

7.5 Página 03 — Pagamentos & Pedidos

Acompanha a evolução dos pedidos ao longo do tempo e a situação de cada um. Usa a tabela de fato junto com as dimensões tempo e status do pedido.

Gráficos:

- **Valor Total por Mês** (colunas): valor de vendas de cada mês, de 1 a 7: cerca de 31 Mil, 28 Mil, 27 Mil, 27 Mil, 30 Mil, 28 Mil e 28 Mil. O valor é estável ao longo dos meses.
- **Soma por Status de Pedidos** (rosca): distribui os 1.524 pedidos entre os status da dim_status_pedido: Entregue (853; 55,97%), Em transporte (292; 19,16%), Processando (147; 9,65%), Cancelado (121; 7,94%) e Devolvido (111; 7,28%).
- **Total Pedidos por mês** (área): quantidade de pedidos de cada mês: 202, 229, 230, 215, 210, 227 e 211, de janeiro a julho. A soma dos meses é 1.524, igual ao total da rosca de status.

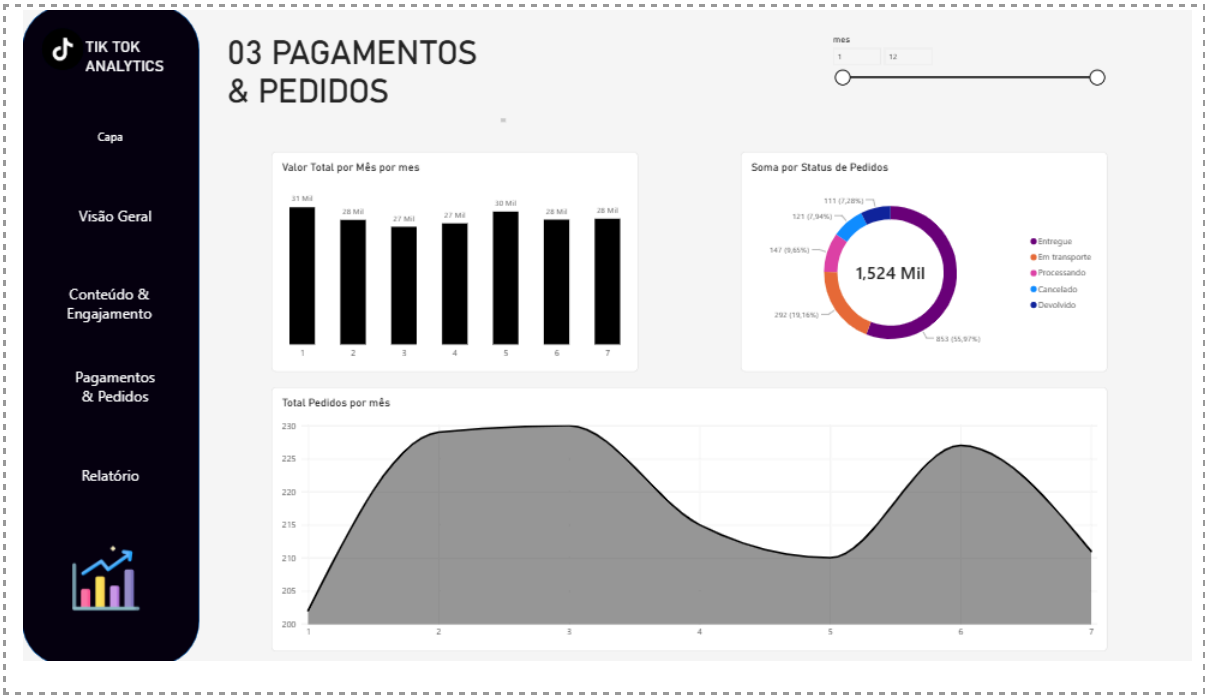


Figura 11 — Página 03 Pagamentos & Pedidos.

7.6 Página 04 — Relatório Geral

Página de fechamento, que reúne em uma única tela os principais indicadores de conteúdo e vendas para apresentação.

Cartões de indicadores (KPIs):

Indicador	Valor exibido
Visualizações	30,35 Bi
Curtidas	3,03 Bi

Indicador	Valor exibido
Vídeos publicados	500

Gráficos, tabela e botões:

- **Pedidos por Hashtag** (linha): volume de pedidos por conjunto de hashtags, em ordem decrescente: #tecnologia #tech, #tiktok #trend, #beleza #dicas, #fitness #treino e #lifestyle #rotina.
- **Média de Produto por Engajamento** (pizza): participação das categorias de produto Moda, Alimentos, Beleza, Casa e Eletrônicos.
- **Tabela por categoria e autor:** mostra categoria, autor, país, views, média da taxa de engajamento e média de likes, com o total geral no rodapé (média de engajamento de 15,09 e média de likes de 6.051.046,61). Permite comparar criadores dentro de cada categoria.
- **Pedidos por Mês** (colunas): quantidade de pedidos de cada mês, de 1 a 7 (202, 229, 230, 215, 210, 227 e 211), a mesma da página 03.
- **Botões de atalho:** “Acessar Plano de Marketing (no Canva)” e “Acessar Documentos (no Google)”, que abrem o plano de marketing e os documentos do projeto, ligando a análise à ação.

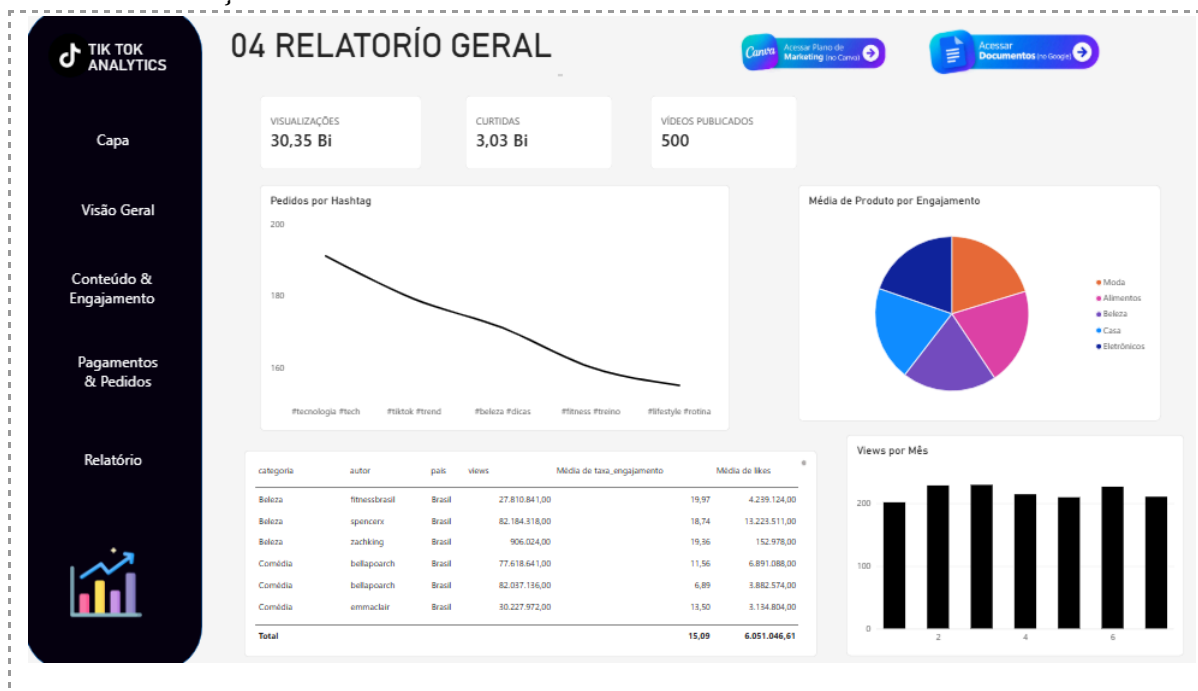


Figura 12 — Página 04 Relatório Geral.

8. Objetivo da modelagem

Reunir em uma única estrutura analítica o caminho completo:

Conteúdo → Engajamento → Vídeo → Pedido → Produto → Pagamento → Status → Data

9. Conclusão

9.1 Resumo do projeto

O projeto percorreu todas as etapas de um fluxo de dados analíticos:

1. **Extração e tratamento:** leitura de CSV (pedidos) e XML (autor/vídeo) no Pentaho, com validação e junção dos fluxos.
2. **ODS:** carga no PostgreSQL em `ods.tik_tok_shop` e classificação dos vídeos em Alto, Médio e Baixo.
3. **Data Warehouse:** modelo estrela no schema `dw`, com quatro dimensões e a tabela fato `fact_pedidos`.
4. **Visualização:** dashboard TIK TOK ANALYTICS no Power BI, com capa e quatro páginas de análise.

O elo entre conteúdo e vendas é o `id_video`, que permite analisar vídeos, criadores, categorias e hashtags junto com valores, status e datas dos pedidos.

9.2 Pontos de atenção

Itens a conferir nas medidas do Power BI:

- **Vídeos sem pedido:** dos 500 vídeos, 89 não têm nenhum pedido. Isso é intencional: os dados foram gerados com alguns vídeos sem compras, para permitir comparar conteúdos que vendem com conteúdos que não vendem. Esses vídeos ficam na `dim_video` e também entram na tabela fato, em linhas com os campos do pedido vazios. Por isso a fato tem 1.613 linhas: 1.524 pedidos mais 89 vídeos sem pedido. Para contar apenas pedidos, use a contagem de `id_pedido`, que ignora as linhas vazias.
- **Período dos dados:** os pedidos do arquivo de origem (`tiktok_shop_pedidos.xml`) vão de 01/01/2026 a 28/07/2026, ou seja, os meses 1 a 7 de 2026. O trimestre 3 tem só o mês de julho (até o dia 28), então não deve ser comparado diretamente com os trimestres 1 e 2. A `dim_tempo` foi criada de 2024 a 2027, e os dados ocupam só parte desse intervalo.